6주차: 의존관계와 백트래킹

선행 작업과 가능한 선택

분석 파이프라인의 작업이 서로를 기다리면 실행이 멈춥니다. 실행 가능한 순서를 찾은 뒤에는 주어진 항목으로 만들 수 있는 선택을 빠짐없이 생성해야 합니다.

남은 선행 조건으로 실행 순서를 정하고, 선택·재귀·복원으로 분기 간 상태를 분리합니다.

6주차 학습 목표

11·12차시: 진입차수·사이클·선택 경로·복원

- 입력 계약과 경계 조건 확인
- 상태 추적과 Python 구현
- 정당성·시간·보조 공간·출력 공간 설명

도입 상황, 표와 점검 문제는 강의용 추가 예시입니다.

11차시 · 유향 사이클과 위상순서

분석 파이프라인 작업의 선행 조건을 만족하는 실행 순서를 구합니다. 작업이 서로를 기다리는 순환을 발견하고 여러 독립 성분도 검사합니다.

선수 지식과 Python 문법

graph=[[] for _ in range(V)]는 독립 목록을 만듭니다. 쌍 [after,before]를 before→after로 바꿉니다. DFS 상태 0/1/2는 미방문/진행 중/완료입니다.

선행 조건과 진입차수

입력 `[after,before]`는 `before→after` 간선입니다.

- `graph[before]`: 완료 후 조건을 줄일 이웃
- `indegree[after]`: 아직 남은 선수 조건 수
- 진입차수 0: 지금 실행 가능한 작업
- 큐가 빈 뒤 처리 수 < V: 사이클 존재

독립된 작업도 초기 큐에 포함합니다.

207 · 문제와 상태

LeetCode 207: Course Schedule · Medium

선수 과목 조건을 모두 만족하며 전체 과목을 끝낼 수 있는지 반환합니다.

`n=3, prerequisites=[[1,0],[2,1]] → True`

before→after로 간선을 만들고 진입차수 0부터 제거합니다. 제거 수가 V이면 True입니다.

207 · 구현

```
def can_finish(n, prerequisites):
    return len(course_order(n, prerequisites)) == n
```

207 · 정당성과 비용

제거 시 남은 선수 조건이 없으므로 선택은 안전합니다. 남은 유향 비순환 그래프에는 진입차수 0 노드가 있어야 합니다. V개를 제거하지 못하면 남은 부분에 사이클이 있습니다.

V=과목 수,E=조건 수. O(V+E) 시간, 인접 리스트 포함 O(V+E) 보조 공간.

210 · 문제와 상태

LeetCode 210: Course Schedule II · Medium

선수 조건을 만족하는 순서 하나를 반환합니다. 불가능하면 빈 목록입니다.

`n=4, prerequisites=[[2,0],[2,1],[3,2]] → [0,1,2,3]`

진입차수 0 큐에서 꺼낸 순서를 저장하고 마지막에 V개인지 확인합니다.

210 · 구현

```
def course_order(n, prerequisites):
    graph, indegree = [[] for _ in range(n)], [0] * n
    for after, before in prerequisites:
        graph[before].append(after)
        indegree[after] += 1
    queue = deque(v for v in range(n) if indegree[v] == 0)
    order = []
    while queue:
        node = queue.popleft()
        order.append(node)
        for neighbor in graph[node]:
            indegree[neighbor] -= 1
            if indegree[neighbor] == 0:
                queue.append(neighbor)
    return order if len(order) == n else []
```

210 · 정당성과 비용

각 과목은 모든 선수가 제거된 뒤에만 큐에 들어갑니다. 따라서 반환 순서에서 모든 before의 위치가 after보다 앞섭니다. 수가 부족하면 전체 순서가 아닙니다.

V=과목 수,E=간선 수. O(V+E) 시간, graph·indegree·queue·출력 합 O(V+E).

상태 추적

제거	indegree(0,1,2,3)	queue 다음
시작	0,0,2,1	0,1
0	0,0,1,1	1
1	0,0,0,1	2
2	0,0,0,0	3
3	0,0,0,0	빈 큐

남은 진입차수가 0이 되는 순간 이웃을 큐에 넣습니다.

실패 입력과 이해 확인

[[1,0],[0,1]]은 사이클입니다. 상태 1인 정점은 현재 탐색 경로에 있습니다. 그 정점으로 돌아가는 간선을 만나면 사이클입니다. 상태 2인 정점은 탐색이 끝났으므로 결과를 재사용합니다.

완료 정점 재방문을 사이클로 보면 어떤 문제가 있나요?

교사용 답은 별도 진행안에 있습니다. 코드 설명은 읽기자료의 구현과 줄별로 대조합니다.

12차시 · 선택·재귀·복원

가능한 조합을 모두 생성하되 순서만 다른 중복은 제거합니다. 한 경로의 선택이 다음 경로에 남지 않도록 공유 상태를 복원합니다.

선수 지식과 Python 문법

path.append(x) 뒤 재귀, 다음 path.pop()으로 마지막 선택만 되돌립니다. result.append(path.copy())는 snapshot입니다. pop()은 마지막에 추가한 선택을 O(1)에 제거합니다. remove(x)는 앞에서부터 같은 값을 찾아 첫 항목을 지우며 최악 O(n)이 듭니다.

78 · 문제와 상태

LeetCode 78: Subsets · Medium

서로 다른 값으로 이루어진 배열의 모든 부분집합을 반환합니다.

`nums=[2,5] → [[],[2],[2,5],[5]]`

start 이상에서 하나를 선택하고 다음 인덱스부터 재귀한 뒤 pop합니다. 매 호출에서 path.copy()를 저장합니다.

78 · 구현

```
def subsets(nums):
    result, path = [], []

    def visit(start):
        result.append(path.copy())
        for index in range(start, len(nums)):
            path.append(nums[index])
            visit(index + 1)
            path.pop()

    visit(0)
    return result
```

78 · 정당성과 비용

부분집합마다 오름차순 인덱스 열이 하나 있습니다. start가 그 열만 허용하므로 모든 부분집합을 정확히 한 번 만듭니다. 복원 후 부모의 path는 호출 전과 같습니다.

n=길이. snapshot 포함 $O(n \cdot 2^n)$ 시간·출력 공간, path와 호출 스택 O(n).

39 · 문제와 상태

LeetCode 39: Combination Sum · Medium

후보를 반복 사용해 target 합을 만드는 중복 없는 조합을 반환합니다.

`candidates=[2,5], target=7 → [[2,5]]`

정렬한 후보에서 같은 index로 재귀해 재사용을 허용하고, remaining보다 큰 후보부터는 중단합니다.

39 · 구현

```
def combinations(candidates, target):
    values, result, path = sorted(candidates), [], []

    def visit(start, remaining):
        if remaining == 0:
            result.append(path.copy())
            return
        for index in range(start, len(values)):
            value = values[index]
            if value > remaining:
                break
            path.append(value)
            visit(index, remaining - value)
            path.pop()

    visit(0, target)
    return result
```

39 · 정당성과 비용

양수 선택마다 remaining이 감소해 종료합니다. 비내림차순 인덱스만 선택하므로 순서만 다른 중복을 피하고 모든 가능한 조합을 표현합니다.

m=후보 수,T=target,c=최소 후보,L=floor(T/c),B=방문 호출 수,S=출력 원소 합. 시간 O(m log m + mB + S), 보조 O(m+L), 출력 O(S). B는 보수적으로 O($(m+1)^{L+1}$).

상태 추적

동작	path	start	저장 결과
visit(0)	[]	0	[]
선택 2	[2]	1	[2]
선택 5	[2,5]	2	[2,5]
pop 5, pop 2	[]	0	snapshot 유지
선택 5	[5]	2	[5]

저장한 경로는 독립 복사본이며 pop은 부모 경로를 복원합니다.

실패 입력과 이해 확인

result.append(path)는 같은 객체를 보관합니다. 마지막 pop 뒤 여러 결과가 빈 목록으로 바뀔 수 있습니다.

재귀에서 return하면 부모 for도 끝나나요?

교사용 답은 별도 진행안에 있습니다. 코드 설명은 읽기자료의 구현과 줄별로 대조합니다.

두 재귀 인덱스의 차이

문제	다음 start	이유
부분집합	index+1	각 원소를 한 번만 사용
조합 합	index	같은 후보 재사용 허용

두 문제 모두 인덱스를 뒤로 돌리지 않아 순서만 다른 중복을 막습니다. 조합 합은 서로 다른 양수 후보가 전제입니다. 0을 허용하면 remaining이 줄지 않는 재귀가 생깁니다.

중간 점검

1. 조건 `[[1,0],[0,1]]`을 만족하는 실행 순서는?
2. 조건 `[[2,0],[2,1]]`에서 `[1,0,2]`도 정답인가요?
3. `[2,5]`의 부분집합 수와 빈 부분집합은?
4. 후보 `[2]`, 목표 4에서 다음 재귀 인덱스는?

독립 실습과 다음 연결

학생 실습

힌트·해설

교사용 진행안

6주차 코드 내려받기

코드·상태 정의·정당성·복잡도·경계 입력을 함께 제출합니다. 8주차에는 주제를 숨긴 혼합 평가가 있습니다.

실행과 참고자료

주차별 코드의 code 폴더에서 실행합니다.

```
python demo_11.py
python demo_12.py
python verify.py
python verify.py --student
```

마지막 명령은 학생 파일을 구현한 뒤 실행합니다.

Erik Demaine·Srini Devadas, MIT 6.006: Introduction to Algorithms (2011).